

面向可微数值组件测试的可执行证据：策略增量与外部缺陷案例

胡宁^{a,*}, 段海涛^b, 李书群^c, 胡初扬^d

^a*School of Mechanical Engineering, Hangzhou Dianzi University, No. 1158, 2nd Street, Qiantang District, Hangzhou 310018, Zhejiang, China*

^b*School of Computer Science, Hangzhou Dianzi University, No. 1158, 2nd Street, Qiantang District, Hangzhou 310018, Zhejiang, China*

^c*School of Mathematics and Physics, Xi'an Jiaotong-Liverpool University, 111 Ren'ai Road, Suzhou Industrial Park, Suzhou 215123, Jiangsu, China*

^d*Department of Computer Science and Engineering, University of California, 1156 High Street, Santa Cruz, CA 95064, USA*

Abstract

可微数值组件需要跨越仿真与学习系统中的软件和硬件接口，但成功调用或存在自动微分路径并不能证明函数值、参数导数和执行策略可靠。本文提出一种可执行测试与资格判定方法。在同一组合成 10 类故障上，四级累积策略分别检出 3、5、8 和 10 类，完整记录检出 200/200 个故障且误拒 0/40 个对照。PyTorch 外部受试对象试验把 240 个注入故障的检出率从 25.00% 提高到 100%。跨 NumPy、SciPy 和 PyTorch 的匹配函数值/性质子集在 252 个注入试验上把检出率从 28.57% 提高到 100%，36/36 个对照均通过。由 SciPy #8906 导出的表示等价性质检出弱样例漏掉的 32/32 个变体。未改动 runner 在官方发布 wheel 中复现 PyTorch #80770 与 SciPy #8906、#15620，并通过当前修复版，形成跨两个项目和三种失效模式的三组完整历史 buggy/fixed 对。结果支持跨受控项目与真实发布版本缺陷的递增可执行证据，但不推断广泛现场发生率。

Keywords: 软件质量, 数值可靠性, 变异测试, 可微编程, 测试判据, 验证与确认

*通讯作者。ORCID: 胡宁 0000-0002-2718-0385; 段海涛 0009-0006-4611-8639; 李书群 0009-0007-9255-018X; 胡初扬 0009-0007-1787-685X。

Email addresses: huning@hdu.edu.cn (胡宁), duanhaitao@hdu.edu.cn (段海涛), Shuqun.Li24@student.xjtlu.edu.cn (李书群), chu207@ucsc.edu (胡初扬)

1. 引言

工程软件把学习模块与传播子、矩阵函数作用、积分步和结构化残差块组合使用。这些组件需要跨越 API、命令行工具、设备、数据类型和持续集成任务。自动微分沿实际计算图求导，却不会消除离散、截断和浮点误差。一个组件可能满足调用接口，但违反数值精度、单位、批量隔离或执行策略要求。

这类问题通常在集成后才暴露，随后带来定位、重新训练和重复系统测试的成本。这里缺少的不是另一个求解器 API，而是一份可移植记录，用来说明测试对象、运行域、执行策略、证据和最终判定。

本文提出一份可执行符合性规范。其模式把运行域和执行请求与函数值、梯度、批量、重复性、OOD、长时域、数据类型与设备、单位、来源、标定和组合证据绑定。稳定的记录接口承载身份、版本、来源、执行和判定字段，后端插件提供特定组件类别所需的数值对照。核心、扩展和应用三个档案允许使用方按照集成边界选择所需证据。

本文贡献包括：(1) 版本化组件模式和三个累积式符合性档案；(2) 支持规范 JSON、请求与实际执行属性对照以及旧记录迁移的参考实现；(3) 在合成故障目录和上游缺陷导出案例上的配对策略比较。本文回答三个实证问题：*RQ1*，数值性质、执行感知和完整证据策略分别增加哪些故障类别；*RQ2*，该递进顺序能否从记录变异迁移到外部数值接口；*RQ3*，表示等价性质能否暴露被单个正确样例掩盖的真实历史缺陷模式。

2. 相关工作与定位

可微编程使数值模拟器和微分方程求解过程能够作为可训练组件使用。代表性方向包括自动微分 [1]、可微物理模拟 [2, 3]、神经常微分方程 [4]、通用微分方程 [5]、物理信息神经网络及其软件生态 [6, 7]，以及学习函数空间映射的神经算子 [8]。这些工作说明了通过科学计算过程反向传播的价值，但并未共同规定单个数值组件在函数值、参数导数、标定和复用方面应满足怎样的接受条件。

现有生态工作在不同层级上。DeepXDE 与 TorchPhysics 组织微分方程模型、训练目标、几何域和参数辨识流程 [7, 9]；SciML/DifferentialEquations 生态则通过统一求解器接口和工作量-精度基准比较算法与应用 [10]。本文以一次实际执行的基元为资格判定单位。在其进入模型或求解器流程之前，协议检查独立函数值/导数参考、数据类型相关门槛和指定组合，并把证据与运行限制写入机器可读记录。

传统工程验证与确认 (V&V) 依据需求、预期用途、现实参照数据、不确定性和生命周期证据评估模型或仿真系统 [11]。本文处理其中更小的单

Table 1: 前序组件论文与本文符合性研究的边界。

工作	主要研究对象	在本文中的作用
DFSC/MLSL [25]	误差受控的 Mittag-Leffler 传播及可微 SciML 接口	一个已实现后端及其独立数值对照
本文	版本化证据记录、累积档案、迁移规则和可执行判定	组件交换规范、标准追溯、覆盖保护和变异验证

元，即实际执行的可微组件。协议检查其前向数值映射、参数导数路径、批量/设备行为及指定组合。组件级 CONFORMANT 判定只说明数值软件层满足声明条件。模型形式、不确定性和实验一致性仍需在端到端 V&V 中评估。

现有标准提供了相邻概念，但没有直接适用于可微数值内核的记录。ISO/IEC 25010 定义软件产品质量特性，ISO/IEC 25023 给出产品质量测量框架 [12, 13]；ISO/IEC/IEEE 29119-2 规定组织级和项目级测试过程，IEEE 1012 处理系统、软件和硬件 V&V [14, 15]。表 2 给出本文组件字段与这些标准的对应关系。该映射建立可追溯的设计依据，并明确本文模式实现的组件级关注点。

数值分析为软件门槛提供诊断依据。前向误差、后向误差、条件性和浮点稳定性对应不同的失效机制 [16]。矩阵指数作用和 Fréchet 导数已有专门的稳定算法 [17, 18]，导数条件数估计可刻画扰动放大 [19, 20]。离散分数阶微积分 [21]、Mittag-Leffler 函数数值评估 [22, 23] 和分数阶 Laplacian 定义 [24] 则为分数阶传播提供参考。本文利用这些既有结果选择独立求值器、扰动探针和接受容差。若两条自动微分路径经过同一个不准确的求值器，仅有梯度一致性仍不足以证明数值正确性。

作者此前的 DFSC 工作提出了误差受控的可微 Mittag-Leffler 传播组件及其 SciML 接口 [25]。该工作研究特定分数阶传播子的可靠求值与求导；本文研究可微组件在交换或进入工作流前应携带哪些证据。MLSL 仅作为四类测试后端之一，本文不把其算法或数值结果作为新贡献。表 1 明确两项工作的边界。报告方式遵循可复现科学计算实践 [26]，并使用 SciPy 等标准科学软件构造独立对照 [27]。

3. 可执行符合性规范

本文所称“符合性”是组件记录满足所选档案及声明门槛的状态。“规范”指带版本的记录模式及其校验规则；“档案”指累积式必需证据集合；“门槛”指预先声明的布尔判据；“证据插件”生成后端特定的测量结果。“测

Table 2: 本文组件记录与现有标准的关系。最后一列给出本文更窄的组件级实现。

来源	相关关注点	本文组件级实现
ISO/IEC 25010	功能适合性、可靠性、性能效率和兼容性	数值门槛、重复性、资源记录和接口等价性
ISO/IEC 25023	软件产品质量测量	显式指标、单位、阈值、样本和观测值
ISO/IEC/IEEE 29119-2	测试计划、监控、执行和完成	冻结档案、可执行检查、规范报告和完成判定
IEEE 1012	基于需求的 V&V 与可追溯证据	声明域、独立参考、来源及需求-证据关联

试动作”执行一项或多项测量，“资格判定”则表示档案级决定。这些术语界定本文实现并验证的判定语义。

每个后端都公开批量映射

$$y = P(x; \theta), \quad (1)$$

并明确输入域、参数域、输出形状、数据类型策略、设备策略和参考求值器。该规范不要求所有基元具有相同数学结构，而要求每个结论都保留其有效域。

每个待测基元保存如下可执行规格

$$\mathcal{C} = (\mathcal{D}_x, \mathcal{D}_\theta, R, \tau, \mathcal{M}, \mathcal{S}, \mathcal{Q}, \mathcal{E}), \quad (2)$$

其中 $\mathcal{D}_x$ 和 $\mathcal{D}_\theta$ 分别为状态域和参数域， R 为独立参考求值器， τ 为门槛容差， $\mathcal{M}$ 记录数据类型、设备和实现元数据， $\mathcal{S}$ 声明已知适用边界， $\mathcal{Q}$ 记录冻结范围和必需覆盖锚点， $\mathcal{E}$ 保存请求与实际执行属性、单位和来源。缺少必需字段的记录可用于诊断，但不符合所选档案。

该映射通过 CQ-01–CQ-10 项目要求实现。补充材料表 S5 从标准关注点追溯到模式字段、测试动作、输出工件和判定规则。这些编号由本文定义，并非所引标准中的条款编号。

3.1. 符合性档案与版本

Core 档案要求函数值与方向梯度精度、批量形状与独立性、重复性、至少 8 个案例，以及标称、边界和异构批量锚点。**Extended** 至少要求 16 个案例，并加入扰动、执行策略、长时域、OOD、数据类型/设备、单位和资源证据。**Application** 至少要求 24 个案例，并加入应用组合锚点、标定和组合证据。候选实现运行前必须冻结范围与采样计划。缺失证据、范围未冻

结或覆盖不足均判为不符合。当前模式为 DFSC-DNC-Conformance-v3。v1 或 v2 迁移保留来源证据，但迁移记录必须重新完成 v3 覆盖要求，不能继承原符合判定。

互操作性在记录边界上定义。参考校验器接受 v3，把 v1 或 v2 显式迁移为待重新判定的记录，并拒绝未知标识。规范 JSON 对对象键排序，采用紧凑分隔符和 ASCII 转义，并在计算 SHA-256 摘要前编码为 UTF-8；数组顺序保留语义。本文验证的互操作范围仅包括 Python API、命令行路径、规范 JSON 和所测 PyTorch 证据插件；JAX、Julia 和独立校验器尚未测试。语义变化必须使用新的模式标识和显式迁移，不能静默解释。

3.2. 可靠性维度

函数值可靠性通过声明的参考结果测量绝对误差、相对误差和有限值率。对于预测值 $\hat{y}$ 和参考值 y^* ，使用 $e_{\text{rel}} = \|\hat{y} - y^*\|_2 / (\|y^*\|_2 + \epsilon)$ ，并采用混合判据

$$\|\hat{y} - y^*\|_2 \leq \tau_{\text{abs}} + \tau_{\text{rel}} \|y^*\|_2. \quad (3)$$

这一判据在参考值接近零时仍然有效。对于归一化参数方向 v ，将自动微分方向导数 $g_{\text{AD}} = \nabla_{\theta} P(x; \theta)^{\top} v$ 与中心差分估计 g_{FD} 比较，并报告 $e_{\text{grad}} = |g_{\text{AD}} - g_{\text{FD}}| / (|g_{\text{FD}}| + \epsilon)$ 。有限差分步长围绕由数据类型确定的标称值进行扫描，而不是只选取一个有利扰动；只有出现稳定一致区间时梯度门槛才通过。参数标定使用带噪观测，报告参数误差、收敛行为、随机种子变化，并在可行时与匹配的经典优化基线比较。模块复用至少包含神经模块和批量算子或物理目标两类组合。OOD、长时域以及批量/设备测试用于暴露边界行为，而不是把它们压缩为一个总分。

式 (3) 的容差必须在审计前声明，并与参考精度、数据类型、条件性和下游任务分辨率对应。实际标定先进行仅含参考解的求值与扰动实验。相对容差应高于参考不确定性与按观测条件性放大后的数据类型舍入误差。绝对容差负责参考值接近零的区域，并且不应小于下游任务真正关心的输出分辨率。float32 与 float64 分别保存容差记录。候选实现运行前冻结所选容差，并通过敏感性表检查阈值的小幅变化是否改变资格状态。对于病态映射，还应记录条件数估计或扰动实验，避免把问题本身的前向敏感性误判为实现缺陷。

每个测试维度保留自己的测量状态；机器可读的档案级判定只有两种。CONFORMANT 表示所有必需证据均存在、范围已冻结、覆盖要求满足且所有门槛在该范围内通过。NONCONFORMANT 表示门槛失败、执行不一致、覆盖失败、迁移后尚未重新判定或缺少证据。适用限制作为显式字段保留。正文可将带有非空限制字段的合格记录表述为“在声明限制下符合”，但该短语不构成第三种机器状态。

本规范将自动微分与数值精度分开处理。自动微分只对实际执行的计算图求导，而近似误差和浮点误差由求值器及其数据类型决定。诊断上可写为

$$e_{\text{grad}} \lesssim \kappa_{P'}(e_{\text{eval}} + e_{\text{AD}} + e_{\text{ref}}), \quad (4)$$

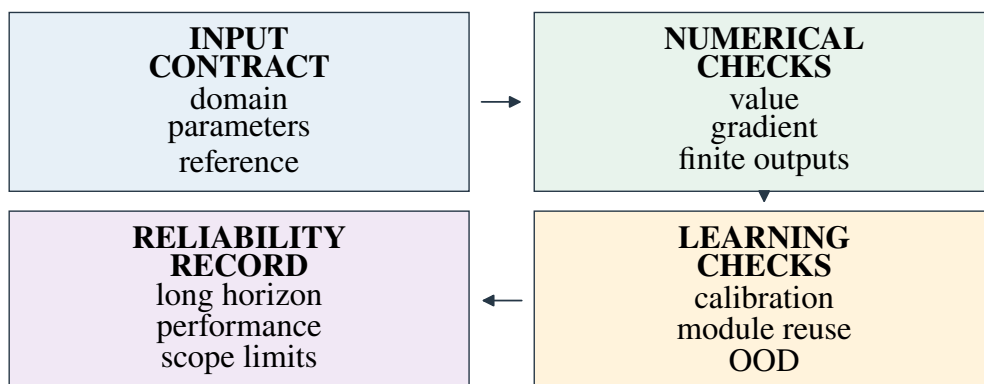
其中三项分别表示执行求值器的近似误差、求导路径误差和独立导数检查误差。令 $z = (x, \theta)$ 且 $G(z) = P'(z)$ ，本文将局部导数敏感性解释为

$$\kappa_{P'}(z; s_z) = \lim_{\delta \rightarrow 0^+} \sup_{\|\Delta z\| \leq \delta s_z} \frac{\|G(z + \Delta z) - G(z)\|}{\delta(\|G(z)\| + \epsilon_0)}, \quad (5)$$

其中 $s_z > 0$ 是预先声明的扰动尺度， $\epsilon_0 > 0$ 是固定的输出尺度保护量。计算该诊断量之前，输入和导数输出需无量纲化，或按记录的物理单位缩放。因此， ϵ_0 与 $\|G(z)\|$ 具有相同的归一化单位；其数值在候选实现运行前根据独立参考的分辨率确定，并写入记录。该量刻画导数映射的条件性；对于矩阵函数，可通过 Fréchet 导数计算或估计 [19, 20]。式 (4) 是诊断分解，不是严格全局上界。

刚性和非正规性需要分别检查。刚性系统中的步进 Jacobian 乘积可能在前向轨迹有界时放大反向敏感性。非正规算子的瞬态增长也可能无法由特征值反映。因此，审计将扰动探针与时域测试配对。对于稳定的 p 阶步进方法，在步长为 h 、步数为 N 时，局部截断误差的累积量级在未考虑具体稳定性放大前可写为 $O(Nh^{p+1})$ 。浮点累积通常与 $O(Nu)$ 同阶，其中 u 为舍入单位，但消去误差和条件数可能改变这一行为 [16]。审计记录数据类型、时域、步长或截断预算、参考精度，以及梯度参考是否沿同一执行路径计算。float32 与 float64 分别保存精度记录。

4. 参考软件架构



Every gate is evaluated on the declared domain

backend | dtype | device | batch | seed | reference | scope limit

CONFORMANT / NONCONFORMANT + explicit scope fields

Figure 1: 组件符合性 workflow。版本化规范把声明域和执行请求与测量证据、未满足要求、运行限制及规范机器可读报告对应起来。

参考实现位于 P4/dfsc_protocol。已有数值测试动作测量有限函数值、函数值误差、方向梯度一致性、批量行为、设备本地性和资源使用。新增的 conformance 模块校验档案，对照请求与实际执行属性，并检查必需证据。canonical_json 消除序列化键顺序差异，record_digest 生成稳定内容标识。同一判定器通过 Python API 和命令行接口调用，输出可用于持续集成门槛和工具交换的规范 JSON。

所有实验采用相同顺序。首先冻结可执行规格及元数据，再在声明域内采样并计算独立参考；随后运行函数值、方向梯度、参数标定和组合测试，并以单因素扩展进行 OOD 和长时域测试。性能测量只针对输出有限且满足数值门槛的配置。最终记录包含汇总统计、最差观测指标、失败样本、执行属性和档案判定。

5. 实验设置

面向标准与接口的验证分为两部分。第一部分使用 40 轮随机记录测试 API-命令行等价性、键顺序不变性、旧证据保留和迁移后强制重新判定。随后使用 10 类变异算子，分别针对截断证据、梯度连接、数据类型、批量隔离、设备本地性、OOD 适用域、单位、来源、范围冻结和最低覆盖要求。每类变异执行 20 次，并以 40 个未修改记录估计误拒。四种预先声明的策

Table 3: 符合性规范要求的证据与接受逻辑。容差按后端和声明域设置；本表统一字段与判定，而非给出通用数值门槛。

门槛	必需观测量	审计记录
函数值	绝对/相对混合误差；有限输出率	容差、参考、类型、适用域
梯度	步长扫描中的方向误差；有限梯度率	稳定一致区间和失败方向
标定	参数误差、损失、收敛和种子变异	优化器、边界、初值、噪声
复用	至少两种可微组合	形状/设备检查和任务指标
压力测试性能	OOD 与长时域误差轨迹同步延迟、吞吐和内存	扩展范围和首次失败区域 硬件、类型、批量、预热、重复次数

略在同一批记录和变异上重放：包络加函数值/梯度检查；增加批量隔离、重复性、OOD 和长时域证据的数值性质策略；继续增加数据类型、设备和单位证据的执行感知策略；以及同时强制来源、范围冻结和档案覆盖的完整记录。比较以故障类别为主要单位。这些最小策略用于隔离各证据层的增量，不代表所有单元测试、性质测试或变形测试实现。第二部分以数值实验生成档案所需测量证据。四个初始后端为 Mittag-Leffler 谱层、矩阵指数作用、定步长线性 RK4 和 Logistic RK4；二维扩展使用可微傅里叶传播子。除特别说明外，多随机种子实验使用 5 个种子。共同神经 OOD 任务使用 3 个种子、512 个训练任务、256 个测试任务、两个目标时刻（1.0 和 1.5）以及两种观测噪声（0.003 和 0.01），运行于 RTX 5070 Laptop GPU。PDE 测试采用从 16^2 到 256^2 的五种网格、五个种子和三个时域，并以独立于 FFT 实现的有限三角模态闭式解作为参考。

外部受试对象试验在三个已安装 PyTorch 接口上重放四级累积策略：标量特殊函数求导、矩阵指数作用和非连续 CUDA 张量收缩。8 个行为故障族、10 个种子和 3 个接口构成 240 个注入试验，另有 30 个未修改对照；24 个“受试接口-故障”组合是异质性分析的簇。历史缺陷分支固定两个 PyTorch 报告和 SciPy #8906 的上游来源与观测。SciPy #8906 先以 32 个源码导出替身变体比较填充表示与数学等价的常规表示，再与 PyTorch #80770、SciPy #15620 一起由未改动 runner 对比官方缺陷 wheel 和当前修复版；最后一个案例检验整数与浮点输入的 `resample_poly` 是否表示同一非零信号，并保存环境、wheel、修复提交、命令、输出与 runner 哈希。

6. 实验结果

各后端指标具有不同的含义和尺度。比较图均按照对应后端预先声明的测试与容差解释；执行栈不同时，时间数据仅作为实现记录。表 5 汇总后续实验如何填充三个累积档案。

6.1. 接口互操作性与故障检出

40 轮随机试验中，Python API 与命令行接口均生成逐字节相同的规范记录，JSON 键重排未改变摘要。旧模式迁移在全部 40 轮中保留原始证据，同时均被正确标记为需要重新判定。变异测试识别了 200/200 个注入故障，汇总检出的 Wilson 95% 区间为 $[0.981, 1.000]$ 。10 类变异均检出 20/20，对应类别下界为 0.839。40 个有效记录均未被误拒，误拒区间为 $[0, 0.088]$ 。这些区间描述有限重复试验的不确定性，不估计真实缺陷发生率，也不证明任意故障均可检出。

配对策略对照进一步定位了增量收益。表 4 显示，随着执行、来源和范围证据被纳入，检出的故障类别由 3 类单调增加到 10 类。四种策略均接受全部 40 个有效记录，因此在该测试集上，覆盖增加没有以降低特异性为代价。

Table 4: 在同一组 10 类故障（每类 20 个变异）和 40 个有效记录上的配对比较。

策略	检出类别	检出故障	误拒有效记录
包络 + 函数值/梯度	3/10	60/200	0/40
数值性质策略	5/10	100/200	0/40
执行感知策略	8/10	160/200	0/40
可执行证据记录	10/10	200/200	0/40

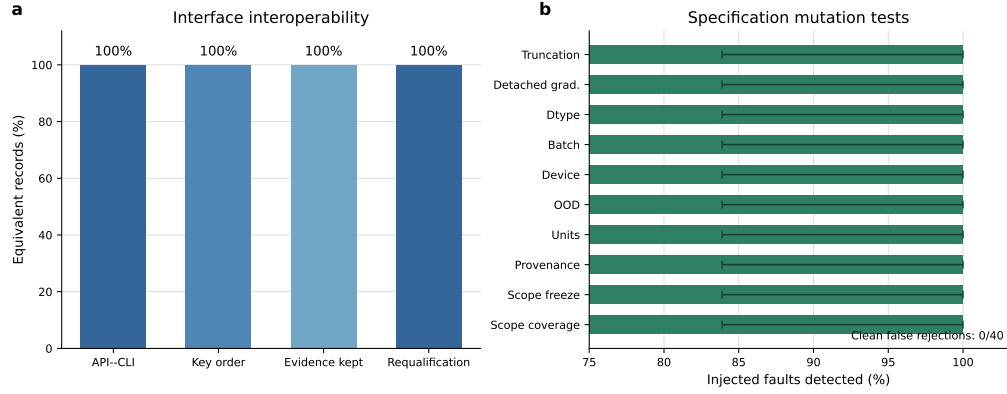


Figure 2: 参考实现验证。(a) API-命令行等价性、键顺序不变性、迁移证据保留及强制重新判定, 各 40 轮; (b) 10 类规范变异, 每类 20 轮, 误差线为 Wilson 95% 区间。有效记录误拒为 0/40。

6.2. 外部接口与历史缺陷导出的策略比较

在 240 个外部受试对象注入试验上, 单点测试的检出率为 25.00%, 加入方向梯度证据后为 73.75%, 数值性质策略为 99.17%, 完整执行证据为 100%; 四种策略均接受 30 个干净对照。在 24 个“受试接口-故障”簇上, 完整策略检出全部簇, 逐层分别新增 3、7、7 和 1 个完全检出簇, 且没有丢失已检出簇。完整策略的留一接口检出率仍为 1.000。这些是一个供应方项目内的受控行为变异, 不是现场缺陷发生率估计。

SciPy #8906 导出的比较在一个历史缺陷家族上回答 RQ3。弱填充样例检出 0/32 个配对变体, 因为该表示恰好把系数放在错误实现硬编码的行; 要求填充表示与常规表示解相等的性质检出 32/32, 当前 SciPy 1.18.0 在 32/32 个对照上满足该性质。对于 PyTorch #80770, 同一 runner (SHA-256 fdb828...014) 在官方 1.11.0+cpu wheel 中返回 $x \log y(0, 2)$ 的零导数; 当前 PyTorch 2.11.0+cu128 返回 $\log 2$ 且保存误差为零。对于 SciPy #8906, 另一未改动 runner (SHA-256 ce901...bc8) 使常规 1×1 表示在官方 1.14.1 wheel 中触发报告的 `IndexError`, 而填充表示给出精确解; 当前 1.18.0 对两种表示均精确求解。对于 SciPy #15620, 第三个 runner (SHA-256 0a7a72...f79) 在官方 1.14.1 中使两种整数类型输出全零, 而浮点参考最大值为 3.001552; 当前 1.18.0 与 float64 参考的最大误差为零。六个角色检查全部通过, 形成跨两个项目的三组完整历史 buggy/fixed 环境对。

随后, 匹配的跨项目子集把七类函数值/执行故障统一应用于 `numpy.linalg.solve`、`scipy.linalg.expm` 和 `torch.lgamma`。在 252 个注入试验上, 单点、完整批量、数值性质和执行证据策略的检出率依次为 28.57%、

57.14%、85.71% 和 100%；36 个干净对照全部通过，完整策略检出全部 21 个“项目-接口-故障”簇。这达到预设的三个受控项目数量，但 NumPy 和 SciPy 分支不检验自动微分，且注入包装器不是现场缺陷。

6.3. 数值门槛覆盖

注册表包含 4 个后端和 6 个要求维度：函数值、梯度、标定、模块复用、OOD 和长时域。我们进一步通过 P1 的公共工厂接口，以稳定混合配置重新运行了 MLSL。在声明的一维 Dirichlet、实负谱参数域内，相对于 80 位高精度参考级数的最大相对函数值误差为 1.32×10^{-12} ；alpha 和 beta 相对于中心差分的梯度误差分别为 2.22×10^{-9} 和 5.00×10^{-10} 。从 (0.90, 2.10) 初值出发，联合标定恢复为 $(\alpha, \beta) = (0.78003, 1.80007)$ 。MLSL 在该运行域内满足六个门槛，同时保留声明的运行限制。四个记录均“在声明限制下符合”；其机器可读状态为 CONFORMANT，运行限制仍是判定记录的一部分。

Table 5: 档案证据与保留的运行限制。三个档案为累积关系；本表概括下文证据类别，不替代具体测量结果。

后端	Core 证据	Extended 证据	Application 证据与记录限制
MLSL	函数值及 α/β 梯度门槛	OOD 与长时域探针	联合标定和混合复用；一维 Dirichlet 问题与实负谱参数
矩阵指数作用	函数值与方向梯度门槛	长时域探针	双参数标定；神经组合为负向任务结果；稳定 2×2 参数盒
线性 RK4 步进	函数值与方向梯度门槛	步长 OOD 和长时域探针	标定与复用；记录步长下的稳定线性系统
Logistic RK4 步进	函数值与方向梯度门槛	步长 OOD 和长时域探针	标定与神经组合；已测试的标量参数及时域

矩阵指数作用在测试时域内保持接近参考精度。两类 RK4 实现在超出声明步长范围后均出现预期的误差增长，其中线性系统的 OOD 误差预算更大。图 3 展示不同种子下的分布；完整压力范围与误差数值见补充表 S2。

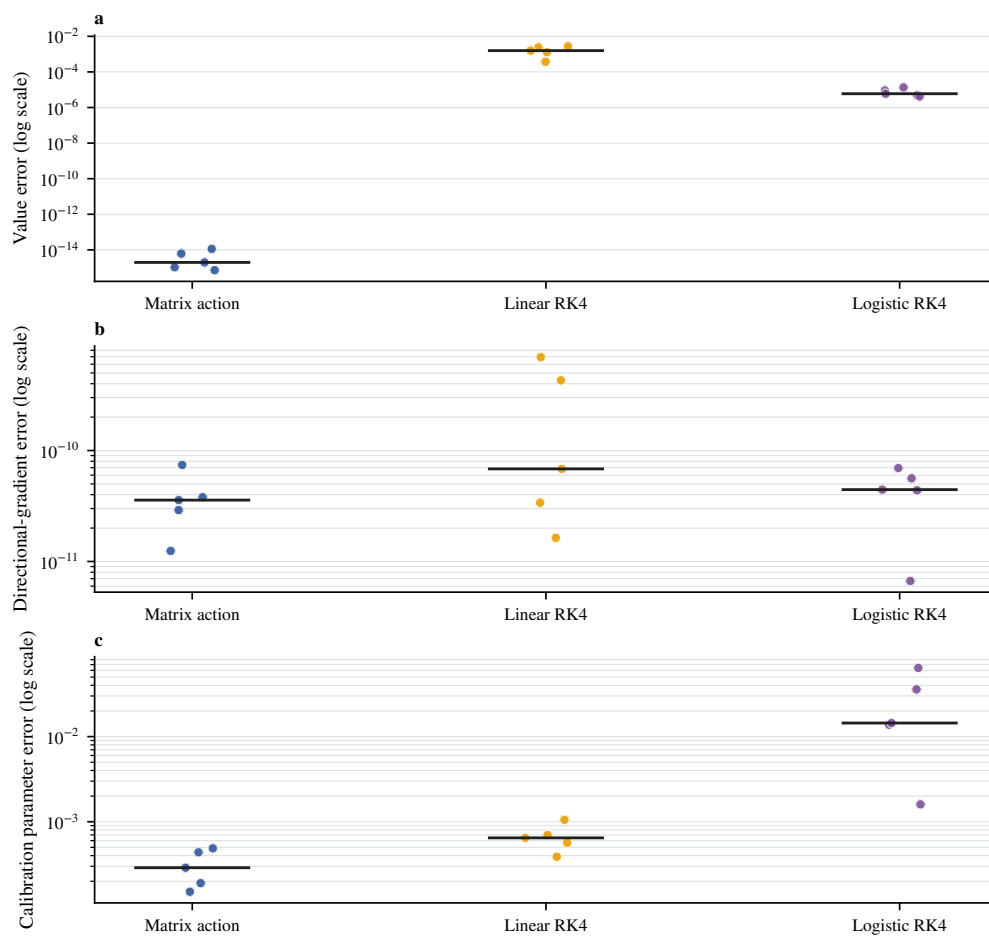


Figure 3: 五个固定随机种子下的后端特定可靠性画像。每个点代表一个种子，水平短线表示中位数。函数值误差分面采用各后端声明的压力指标（矩阵指数作用采用长时域最大误差，两类 RK4 步进采用 OOD 最大误差）；其余分面使用相同的方向梯度和标定定义。所有坐标轴均为对数尺度。

同一组记录还覆盖重复仿真调用。我们在稳定线性系统 $y'(t) = Ay(t)$ 上考察 $t = 1, 2, 4, 8, 12$ ，并让精确矩阵指数作用与定步长 RK4 使用相同初值和终止时刻。五个随机初值下，矩阵指数作用保持在参考精度附近，RK4 误差则随步长减小而下降（图 4）。

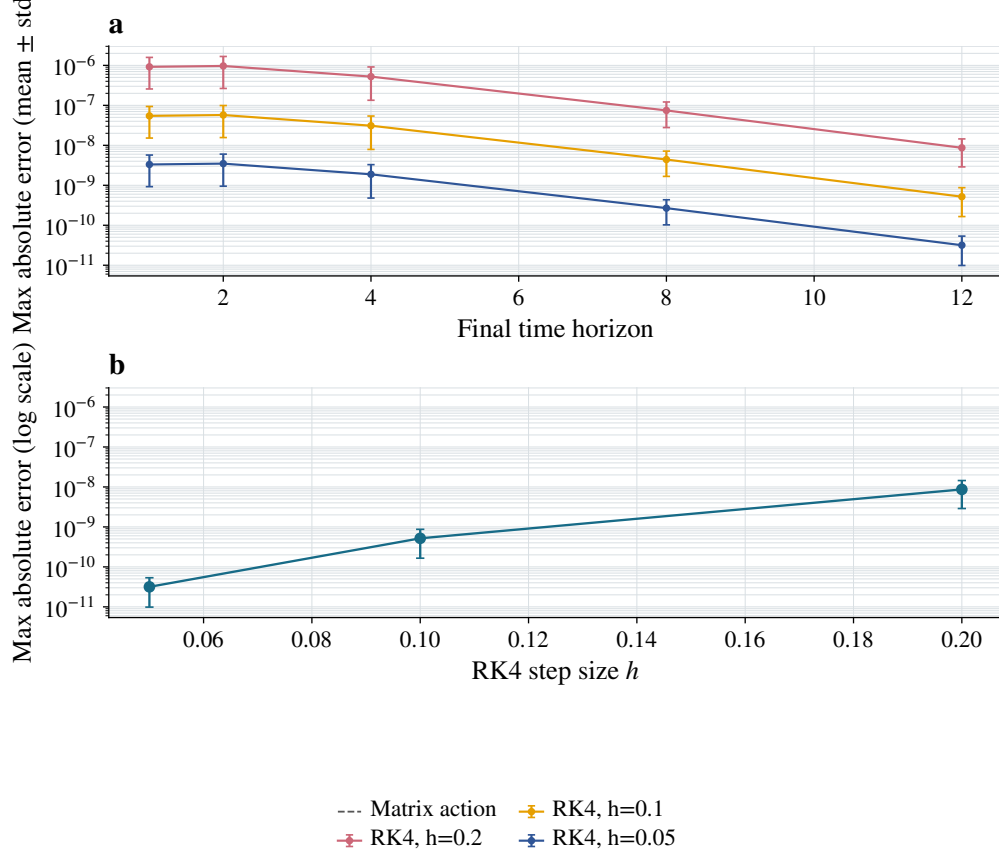


Figure 4: 固定稳定 2×2 线性系统上的轨迹与步长证据。左图报告不同终止时刻的轨迹误差；右图报告 $t = 12$ 时 RK4 的步长敏感性。误差线为五个随机初始状态的标准差。

作为优化器层面的检查，我们在同一个稳定 2×2 矩阵指数作用和同一组噪声观测上比较 Adam 与 L-BFGS-B[28]，使用 5 个固定随机种子，并保持参数边界和初值一致。两种方法在匹配设置下得到相近的参数误差与最终损失。图 5 展示逐种子的配对恢复；逐种子数值、工作量和执行时间见补充表 S1。

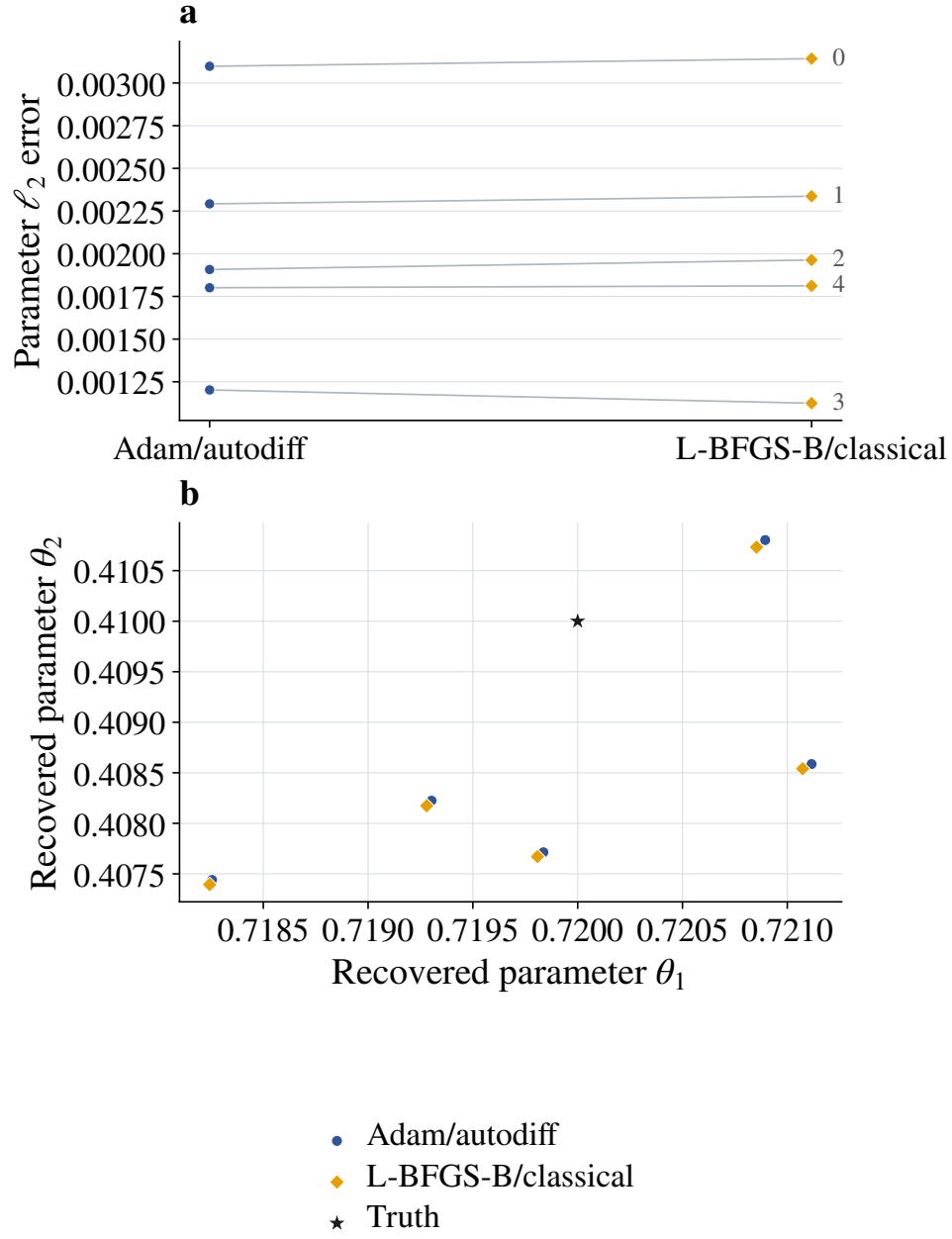


Figure 5: 匹配双参数标定的逐种子可复现性。左图配对显示 Adam/自动微分与 L-BFGS-B/经典优化在相同噪声观测下的参数误差；右图显示恢复参数相对于真实值的位置，连线标识相同种子。

6.4. 任务级迁移与 OOD 证据

我们进一步在两个具有来源记录的公开数据集上评估结构化基元。对于 GeoTES 中试规模热电偶数据 (Zenodo DOI 10.5281/zenodo.18979098), 使用首个实测充放热周期进行标定, 并在第二周期进行迁移测试。三个随机种子下, DFSC 加残差模型的第二周期相对误差为 0.3619 ± 0.0085 , 纯 MLP 为 0.3632 ± 0.0071 , 参数量分别为 750 和 1251。对于加热蒸汽注入温度剖面 (Zenodo DOI 10.5281/zenodo.15064388), 留出工况 RMSE 分别为: 直接 DFSC 43.33 ± 0.56 K, 混合模型 16.03 ± 0.14 K, 纯 MLP 13.25 ± 1.45 K。后一结果作为负向任务级证据保留: 可靠的可微基元并不意味着在所有数据集上都具有无条件的预测优势。在 Application 档案中, 这两组数据用于检验任务证据和运行限制能否与组件级符合性写入同一记录。负向结果具有直接的判定意义: 它阻止使用者在缺少匹配任务证据时, 把数值组件符合性误报为预测模型优越性。

为区分观测密度与外推时域的影响, 我们进一步在首个完整周期内进行均匀抽样, 并固定第二周期作为留出集。在保留 40% 观测时, 混合模型的误差为 0.3607 ± 0.0082 , 纯 MLP 为 0.3657 ± 0.0084 , 参数量仍分别为 750 和 1251。在 20%、60% 和 100% 观测比例下, 混合模型未优于纯 MLP。这里观测到的收益仅出现在中等数据预算下; 图 6 汇总了两个公开数据对比。

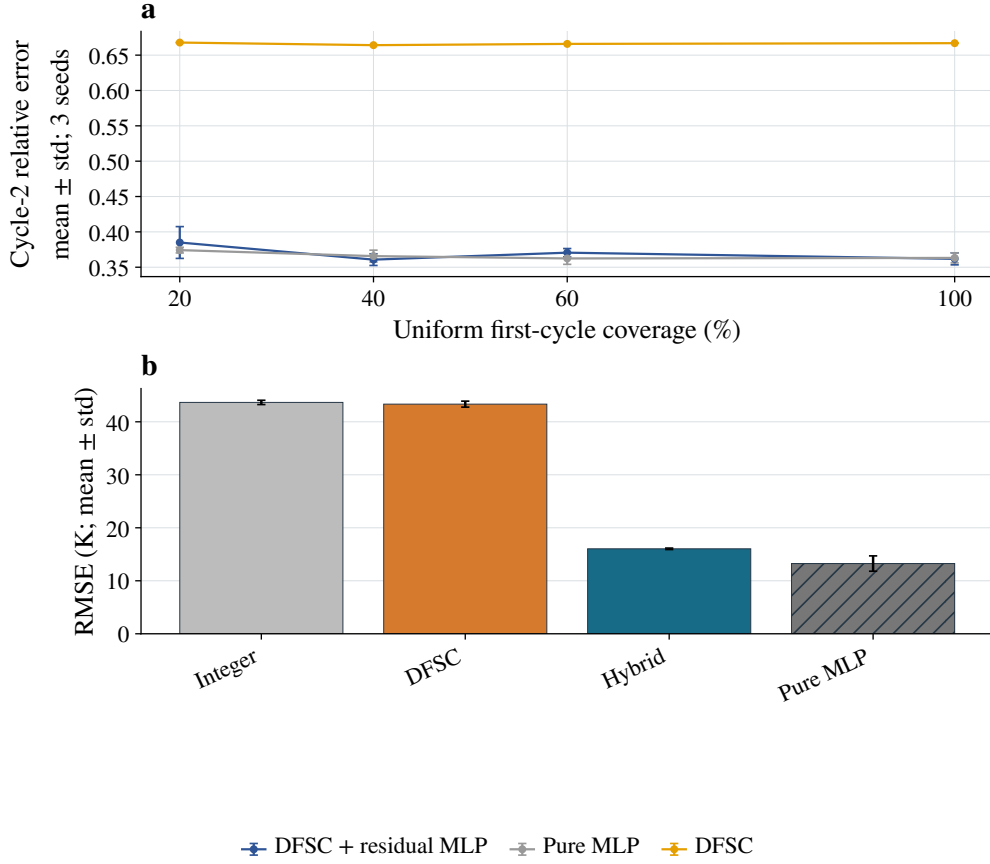


Figure 6: 公开数据迁移证据。GeoTES 分面报告首个完整周期均匀观测比例下的跨周期误差；加热蒸汽分面报告留出工况 RMSE。误差线为三次随机种子实验的标准差。加热蒸汽中的负向比较被保留，用于说明基元可靠性不等同于所有任务上的普遍预测优势。

模块级 *OOD* 证据。在共同神经任务中，Logistic 基元在两个目标时刻和两种噪声下均改善 *OOD* RMSE。例如在 $t = 1.5$ 、噪声 0.003 时，基元模型为 0.2340 ± 0.0323 ，纯 MLP 为 0.2713 ± 0.0096 。矩阵基元没有改善该任务：同一设置下分别为 0.1998 ± 0.0272 和 0.1638 ± 0.0097 。图 7 给出完整匹配对比。当前实现中基元模型训练也更慢。匹配结果表明，组合收益取决于具体任务及其潜变量结构能否被辨识。

矩阵基元的负向结果并非来自矩阵指数求值器失效。在该任务中，结构化模型需先从稀疏轨迹快照推断每个样本对应的 2×2 生成矩阵 A ，再通过 $\exp(tA)y_0$ 预测状态。纯 MLP 则直接预测目标状态。结构化路径在状

态预测之外还引入了潜在参数辨识问题。若 $A = V\Lambda V^{-1}$ 可对角化，则

$$\|\exp(tA)\| \leq \kappa(V) \exp\left(t \max_i \operatorname{Re} \lambda_i\right), \quad (6)$$

弱衰减模态和病态特征向量基可能放大生成矩阵的推断误差；对于非正规矩阵，仅考察特征值还不足以刻画瞬态增长。我们在 12 个种子-时刻-噪声配置上重复结构化矩阵实验，并将逐样本 OOD 误差与生成矩阵诊断量相关联。特征向量条件数对数与状态误差对数的平均 Pearson 相关为 -0.044 ，平均 Spearman 相关为 -0.050 。特征向量条件性本身不能解释该负面结果。非正规性与潜在参数误差的平均正相关分别为 0.305 和 0.290 。

我们还检验了一个可在外推前计算的局部可观测性指标。对于三个观测状态，构造 $[\exp(t_1 A)y_0; \exp(t_2 A)y_0; \exp(t_3 A)y_0]$ 关于 A 的四个元素的 Jacobian。其最小奇异值和条件数均未给出预期的单调预警：最小奇异值对数与状态误差对数的平均 Pearson 和 Spearman 相关分别为 0.46 和 0.50 ，Jacobian 条件数对数与状态误差对数的平均 Pearson 相关为 -0.19 。这些数据不支持采用单一标量阈值。审计据此同时记录局部可观测性、扰动敏感性、非正规性和参数拟合诊断；不利组合只触发定向扰动测试和匹配基线测试，而不直接判为拒绝。函数值与梯度通过门槛，只能确认求值器在声明域内可靠，不能解决辨识不足的潜在反问题。

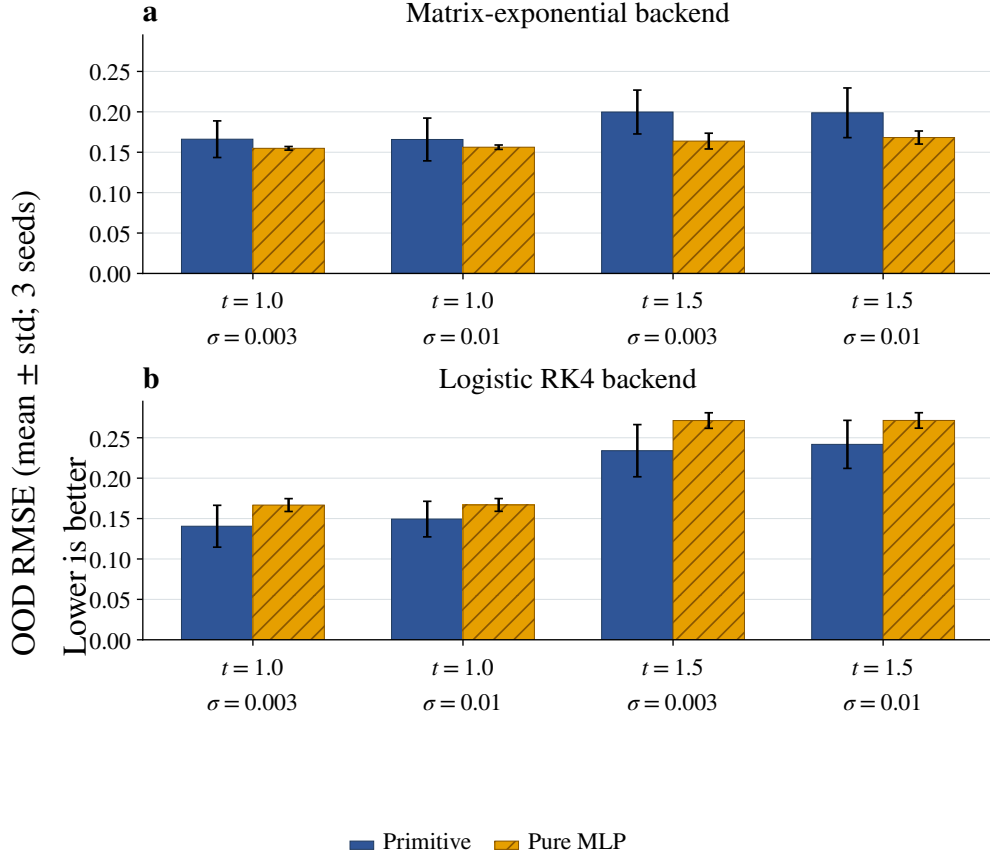


Figure 7: 匹配的模块级 OOD 任务。两个分面分别显示矩阵和 Logistic 后端；柱高为三次随机种子实验的均值，误差线为标准差，RMSE 越低越好。Logistic 基元在测试设置下改善纯 MLP 控制，而矩阵基元并未普遍优于纯 MLP。

6.5. 执行与系统级审计

在 RTX 5070 Laptop GPU 上，三个通用后端在 batch 为 1、64、256 和 1024、float64 条件下均产生有限输出。吞吐随 batch 增大而提升（图 8）。性能记录同时保存预热次数、同步计时、峰值显存、数据类型和输出形状，以便未来后端按同一格式评估。

我们还以 float32 和 float64 重复执行 batch 1024 的矩阵指数作用与 256^2 周期热传播。float32 的函数值误差为 10^{-7} 量级，float64 则降至 10^{-15} 量级。float32 的增量 CUDA 显存约为 float64 的一半，但在这两个受内核启动开销影响的任务中，同步延迟仅变化约 3%。图 9 汇总该权衡；完整的函数值误差、梯度误差、延迟和显存测量见补充表 S3-S4。门槛按数据类型和

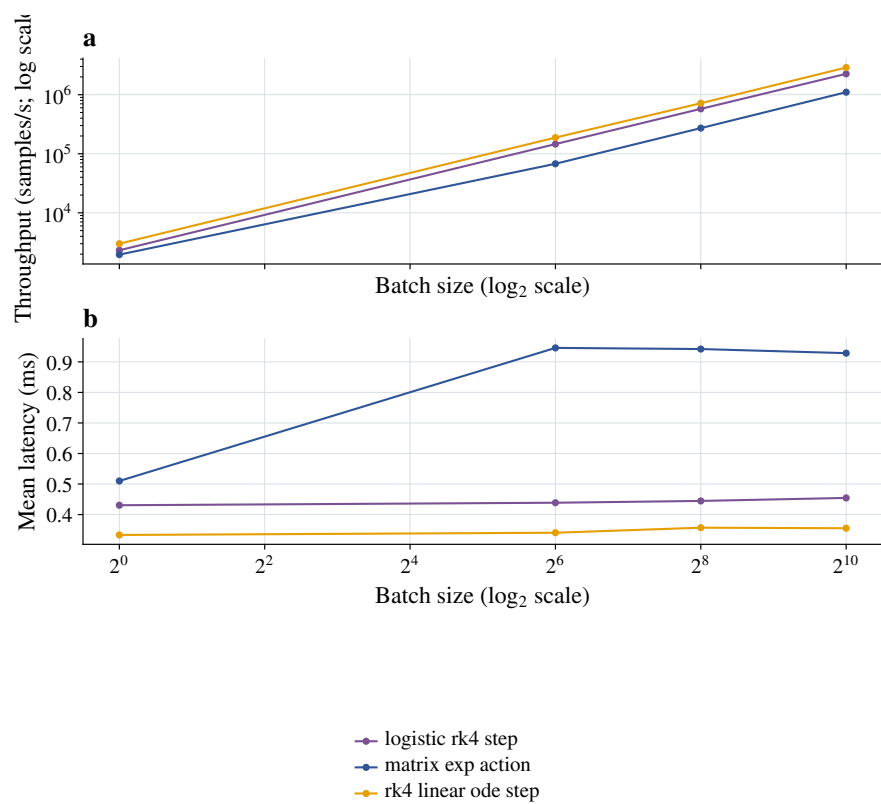


Figure 8: RTX 5070 Laptop GPU 上的批量/设备性能。左图报告吞吐量, 右图报告 float64、50 次迭代下的平均延迟。

参考精度分别标定。

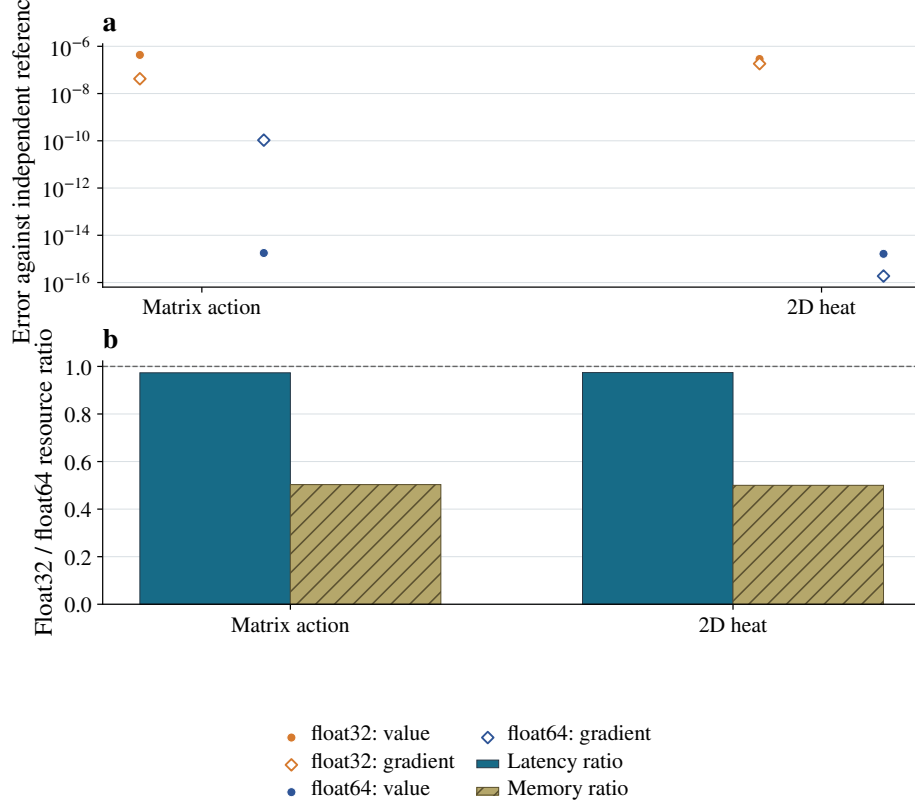


Figure 9: RTX 5070 Laptop GPU 上不同数据类型的精度与资源权衡。图 (a) 给出相对于独立参考的函数值及参数梯度误差；图 (b) 给出 float32 相对 float64 的延迟与增量显存比。

直接二维 PDE 审计。我们将同一协议进一步用于 $[0, 2\pi]^2$ 周期域上的热方程 $u_t = \kappa \Delta u$ 。基元计算傅里叶乘子 $\exp[-\kappa(k_x^2 + k_y^2)t]$ ，有限解析模态展开则为场值和 κ 导数提供独立于 FFT 实现的闭式参考。在从 16^2 到 256^2 状态、 $t = 0.1$ 到 1.0 的 75 个网格-种子-时域配置中，最大场值绝对误差的最差值为 6.99×10^{-15} ，能量梯度相对误差的最差值为 4.75×10^{-16} ，所有输出和梯度均为有限值。在 RTX 5070 Laptop GPU 上，测试的单场调用中位前向延迟为 0.369–0.397 ms，峰值增量显存由 0.011 MiB 增至 2.50 MiB。该规模下近似平坦的时间曲线主要受内核启动开销支配，不能外推为 FFT 的渐近复杂度；显存则随空间自由度增长，相关误差与资源曲线见图 10。

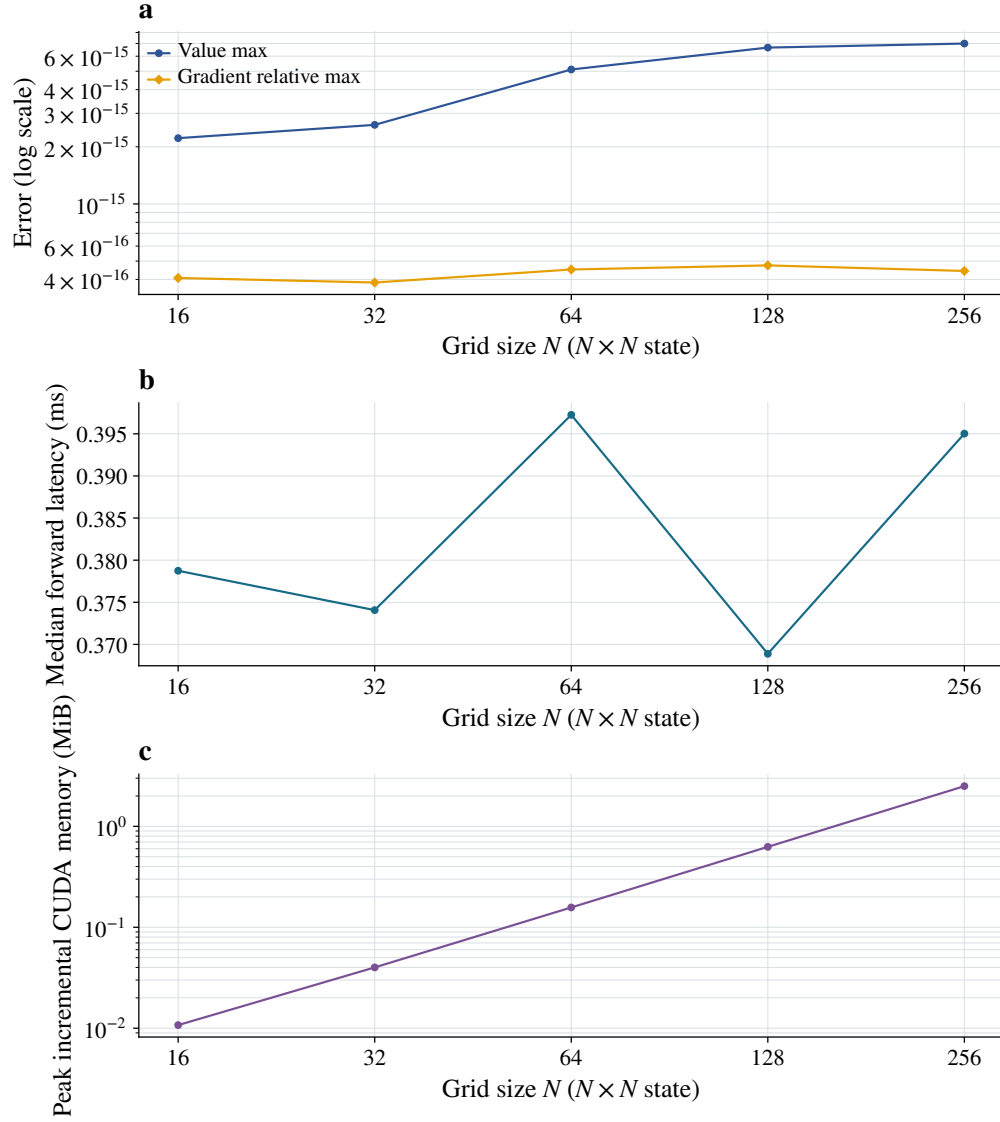


Figure 10: 二维周期热方程直接审计。左图报告五个种子和三个时域中的最差场值及参数梯度误差；中图和右图分别报告同步中位前向延迟与峰值增量 CUDA 显存。闭式模态展开独立于 FFT 实现。

7. 讨论

标准映射明确了本文方案在数值分析与软件测试过程之间的位置。ISO/IEC 25010 和 25023 提供质量特性与可测字段依据；ISO/IEC/IEEE 29119-2 对应冻结测试过程与完成判定；IEEE 1012 对应从需求到独立证据的可追溯性。这些映射共同为组件级规范及其可执行测试提供可追溯依据。

变异结果表明，当前规则识别了实验所表示的 10 类故障，其中包括范围未冻结和档案覆盖不足。配对策略对照给出了更具体的结论：包络加函数值/梯度策略仍不能发现批量、OOD、执行策略和冻结范围缺陷；逐层加入相应证据后，这些预先声明的故障类别被检出，且有效对照上的误拒没有增加。该结论只针对冻结的故障目录，不表示完整记录可以替代所有测试方法。Wilson 区间只描述本实现与所选变异模型的重复试验，不估计真实缺陷发生率或证明检出完备性。后续变异应覆盖畸形单位、对抗性来源、部分设备回退、随机非确定性和跨版本语义漂移。还需独立实现检验模式本身是否足够明确，而不是依赖本参考代码的共同假设。

外部受试对象顺序以及完整 PyTorch #80770 与 SciPy #8906/#15620 对提高了真实性，但没有闭合外部有效性。匹配子集已达到三个独立维护项目，但每个项目只贡献一个接口，NumPy/SciPy 分支不含自动微分。重复种子和表示仍只计作家族内试验，不能计作独立项目或缺陷。三个真实家族现已覆盖错误导数、索引异常和静默 dtype 相关输出，消除了此前只有两个家族的限制；但它们仍只跨两个项目，面向 JSS 的现场有效性主张还需要更广泛的历史复现。

轨迹实验把组件测试延伸到重复模拟。函数值和梯度审计刻画一次调用；重复应用还取决于模拟时域和步长误差预算。图 4 量化了受控传播子中的这一依赖关系。非线性、刚性和复杂几何系统需要相应的轨迹测试，才能赋予相同的资格状态。当前实验覆盖低维受控组件、一个结构化二维周期 PDE、两个公开数据迁移测试和 PyTorch 执行，说明同一协议能够用于不同数值结构。尚未覆盖变阶或分布阶算子、非线性或刚性 PDE、复杂几何及非结构网格。向这些问题扩展时需要补充四类信息：在声明域中记录网格与几何元数据；以构造解、网格加密或不同离散方法建立独立对照；记录非线性求解和预条件器的收敛行为；设置随时域变化的函数值、梯度和守恒门槛。上述扩展应形成新的资格测试，不能直接沿用本文的 CONFORMANT 判定。JAX 和 Julia 后端也需重新验证设备与自动微分行为。当前 2×2 矩阵实验揭示了弱可辨识性与非正规性的共同影响，但尚不能量化其在更大系统中的增长。

对于更高维模拟，审计预算必须随离散规模增长。设状态维数为 n 、批量大小为 B 、函数值/参考样本数为 K 、方向梯度探针数为 q 。若一次基元和参考求值的成本分别为 $C_P(n)$ 与 $C_R(n)$ ，则主要工作量约为

$K[C_P(n) + C_R(n)] + q[C_{P,AD}(n) + 2C_R(n)]$ 。

方向探针可避免显式构造完整参数 Jacobian。活动状态至少需要 $O(Bn)$ 存储，反向计算图的额外开销取决于基元实现。可执行规格记录分块大小、稀疏结构、检查点策略和峰值显存。

独立参考可能成为主要成本。稀疏有限元基元可以采用 Krylov 作用，而对照计算可能需要网格加密、高阶单元或不同离散方法。非线性和刚性问题还会增加收敛非线性求解、Jacobian 或预条件器构造及轨迹级参考计算。无网格基元的成本还取决于配点数量和残差导数阶数。

缺少闭式解时，可采用构造解、守恒平衡和加密研究。若参考沿用同一离散路径，则必须标记为非独立。周期热方程实验已在 65,536 个空间状态上记录网格规模、函数值误差、梯度误差、同步延迟和峰值显存。Navier-Stokes、刚性多物理场、非周期边界和非结构有限元网格仍需单独开展资格测试。

7.1. 向其他组件类别迁移及标准化路径

本文可迁移的是记录边界，而不是一套适用于所有软件的数值门槛。数据库内核、仿真模块和 AI 推理组件可以复用模式身份、版本迁移、来源、请求与实际执行字段、规范序列化及资格结果；但每一类组件仍需建立相应的证据插件、独立对照、容差、变异算子和运行域元数据。本文实验只对所研究的四类数值后端进行资格判定，并未给出其他组件类别的符合性条件。

研究规范若要发展为共识标准，需要形成规范性字段定义和公开符合性样例，获得至少一个独立实现，并开展跨组织轮转测试。正式标准化还需规定受控词汇、版本弃用、安全与来源要求的治理方式，以及处理不兼容解释的程序。本文发布的模式与复现材料可为此类评估和互操作研究提供可执行输入。

7.2. SciML 流程中的组件选择

当参数能够由现有观测辨识、独立参考能够覆盖声明域，且函数值和梯度门槛在目标时域内保持稳定时，结构化基元是合理选择。任务级收益仍应与非结构化控制模型比较。

观测稀疏或噪声过大时，潜在反问题可能欠定。非正规或刚性动力学会放大瞬态误差和反向 Jacobian 乘积。同路径参考提供的证据也弱于独立构造的参考。出现这些情况时，记录保存适用限制；若所选档案的必需门槛不满足，则判为 NONCONFORMANT，并报告扰动与可辨识性诊断。门槛未通过属于符合性结果，不是软件异常。

8. 可复现性与工程软件可用性

实验脚本、JSON 结果、模式校验器、软件测试和论文数据构建脚本均位于公开 DFSC 仓库的 P4 目录。符合性、合成策略、外部受试对象和历史缺陷导出记录均保存在 P4/results/；API 与命令行仅共享判定器，不共享序列化路径。仓库保存了复现所需的实现、输入、来源和声明边界。

9. 结论

本文提出并实现了一种面向可微数值组件的可执行测试与资格判定方法。在配对的 10 类故障目录上，四级策略分别检出 3、5、8 和 10 类；同一顺序在 240 个外部 PyTorch 试验上获得 25.00%、73.75%、99.17% 和 100%。匹配的 NumPy/SciPy/PyTorch 子集在 252 个试验上获得 28.57%、57.14%、85.71% 和 100%，达到三个受控项目门槛。由 SciPy #8906 导出的表示等价性质检出被弱样例漏掉的 32/32 个变体。未改动 runner 还区分了 PyTorch #80770 与 SciPy #8906/#15620 的官方缺陷版和当前修复版，形成跨两个项目的三组完整历史版本对。因此，本文把递增证据连接到外部接口和真实发布版本缺陷，同时仍只提供可证伪的组件级论证，而不是普适可靠性保证。

资助

本研究得到以下项目资助：上海交通大学海洋工程国家重点实验室，项目编号 GKZD010089；中国科学院声学研究所声学国家重点实验室，项目编号 SKLA202406。资助方未参与研究设计、数据收集与分析、结果解释、稿件撰写或投稿决定。

数据可用性

符合性实现、数值实验、生成的 JSON 报告、测试、论文数据构建器和制图流程均已公开于 DFSC 仓库带标签的 P4 目录：github.com/hzhooning-art/DFSC/tree/p4-jss-submission-v1.1/P4。面向标准与接口的互操作和变异测试数据保存在 P4/results/p4_csi_conformance_validation.json。开源数据集的仓库 DOI 已在相应实验中引用；配套 Zenodo 记录发布后将补入软件归档 DOI。

CRediT 作者贡献声明

胡宁：概念化、方法学、软件、验证、形式化分析、调查、数据整理、可视化、初稿撰写、审阅与编辑。段海涛：方法学、软件、验证、调查、审阅与编辑。李书群：验证、调查、审阅与编辑。胡初扬：验证、调查、审阅与编辑。

利益冲突声明

作者声明不存在可能影响本文研究工作的已知经济利益冲突或个人关系。

稿件准备过程中生成式人工智能与人工智能辅助技术的使用声明

本文准备过程中，作者使用 OpenAI Codex 辅助语言编辑、稿件结构整理、一致性检查以及代码和文档审阅。使用该工具后，全体作者根据需要对相关内容进行了审核和修改，并对发表内容承担全部责任。该工具未用于生成或伪造实验数据。

致谢

本研究使用了杭州电子科技大学的计算资源。

References

- [1] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, J. M. Siskind, Automatic differentiation in machine learning: A survey, *J. Mach. Learn. Res.* 18 (2018) 1–43. URL: <https://jmlr.org/papers/v18/17-468.html>.
- [2] Y. Hu, L. Anderson, T.-M. Li, Q. Sun, N. Carr, J. Ragan-Kelley, F. Durand, DiffTaichi: Differentiable programming for physical simulation, in: *International Conference on Learning Representations*, 2020. URL: <https://openreview.net/forum?id=SVQXoincM3>.
- [3] P. Holl, N. Thuerey, V. Koltun, Learning to control PDEs with differentiable physics, in: *International Conference on Learning Representations*, 2020. URL: <https://openreview.net/forum?id=HyeSin4FPB>.

- [4] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, D. K. Duvenaud, Neural ordinary differential equations, in: *Advances in Neural Information Processing Systems*, volume 31, 2018, pp. 6571–6583. URL: <https://proceedings.neurips.cc/paper/2018/hash/69386f6bb1dfed68692a24c8686939b9-Abstract.html>. arXiv:1806.07366.
- [5] C. Rackauckas, Y. Ma, J. Martensen, C. Warner, K. Zubov, R. Supekar, D. Skinner, A. Ramadhan, A. Edelman, Universal differential equations for scientific machine learning, arXiv preprint arXiv:2001.04385 (2020). doi:10.48550/arXiv.2001.04385.
- [6] M. Raissi, P. Perdikaris, G. E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *J. Comput. Phys.* 378 (2019) 686–707. doi:10.1016/j.jcp.2018.10.045.
- [7] L. Lu, X. Meng, Z. Mao, G. E. Karniadakis, DeepXDE: A deep learning library for solving differential equations, *SIAM Rev.* 63 (2021) 208–228. doi:10.1137/19M1274067.
- [8] N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, A. Anandkumar, Neural operator: Learning maps between function spaces with applications to pdes, *J. Mach. Learn. Res.* 24 (2023) 1–97. URL: <https://www.jmlr.org/papers/v24/21-1524.html>.
- [9] D. N. Tanyu, et al., Deep learning methods for partial differential equations and related parameter identification problems, *Inverse Probl.* 39 (2023) 103001. doi:10.1088/1361-6420/ace9d4.
- [10] C. Rackauckas, Q. Nie, Confederated modular differential equation APIs for accelerated algorithm development and benchmarking, *Adv. Eng. Softw.* 132 (2019) 1–6. doi:10.1016/j.advengsoft.2019.03.009.
- [11] National Aeronautics and Space Administration, Standard for Models and Simulations, Technical Report NASA-STD-7009B, National Aeronautics and Space Administration, 2024. URL: <https://standards.nasa.gov/standard/nasa/nasa-std-7009>.

- [12] International Organization for Standardization, ISO/IEC 25010:2023: Systems and Software Engineering—Systems and Software Quality Requirements and Evaluation (SQuaRE)—Product Quality Model, Technical Report, ISO, 2023. URL: <https://www.iso.org/standard/78176.html>.
- [13] International Organization for Standardization, ISO/IEC 25023:2016: Systems and Software Engineering—Systems and Software Quality Requirements and Evaluation (SQuaRE)—Measurement of System and Software Product Quality, Technical Report, ISO, 2016. URL: <https://www.iso.org/standard/35747.html>.
- [14] International Organization for Standardization, ISO/IEC/IEEE 29119-2:2021: Software and Systems Engineering—Software Testing—Part 2: Test Processes, Technical Report, ISO, 2021. URL: <https://www.iso.org/standard/79428.html>.
- [15] IEEE Standards Association, IEEE 1012-2024: Standard for System, Software, and Hardware Verification and Validation, Technical Report, IEEE, 2024. URL: <https://standards.ieee.org/ieee/1012/12536/>.
- [16] N. J. Higham, Accuracy and Stability of Numerical Algorithms, 2 ed., Society for Industrial and Applied Mathematics, Philadelphia, 2002. doi:10.1137/1.9780898718027.
- [17] A. H. Al-Mohy, N. J. Higham, Computing the action of the matrix exponential, with an application to exponential integrators, SIAM J. Sci. Comput. 33 (2011) 488–511. doi:10.1137/100788860.
- [18] A. H. Al-Mohy, N. J. Higham, Computing the frechet derivative of the matrix exponential, with an application to condition number estimation, SIAM J. Matrix Anal. Appl. 30 (2009) 1639–1657. doi:10.1137/080716426.
- [19] N. J. Higham, Functions of Matrices: Theory and Computation, Society for Industrial and Applied Mathematics, Philadelphia, 2008. doi:10.1137/1.9780898717778.
- [20] N. J. Higham, S. D. Relton, Estimating the condition number of the frechet derivative of a matrix function, SIAM J. Sci. Comput. 36 (2014) C617–C634. doi:10.1137/130950082.

- [21] C. Lubich, Discretized fractional calculus, *SIAM J. Math. Anal.* 17 (1986) 704–719. doi:10.1137/0517050.
- [22] R. Garrappa, Numerical evaluation of two and three parameter mittag-leffler functions, *SIAM J. Numer. Anal.* 53 (2015) 1350–1369. doi:10.1137/140971191.
- [23] R. Gorenflo, A. A. Kilbas, F. Mainardi, S. V. Rogosin, *Mittag-Leffler Functions, Related Topics and Applications*, Springer, Berlin, 2014. doi:10.1007/978-3-662-43930-2.
- [24] A. Lischke, G. Pang, M. Gulian, F. Song, C. Glusa, X. Zheng, Z. Mao, W. Cai, M. M. Meerschaert, M. Ainsworth, G. E. Karniadakis, What is the fractional laplacian? a comparative review of ten definitions, *J. Comput. Phys.* 404 (2020) 109009. doi:10.1016/j.jcp.2019.109009.
- [25] N. Hu, H. Duan, S. Li, C. Hu, DFSC: Error-controlled differentiable mittag-leffler propagation for fractional scientific machine learning, 2026. doi:10.48550/arXiv.2607.29038. arXiv:2607.29038, arXiv preprint arXiv:2607.29038.
- [26] G. Wilson, J. Bryan, K. Cranston, J. Kitzes, L. Nederbragt, T. K. Teal, Good enough practices in scientific computing, *PLOS Comput. Biol.* 13 (2017) e1005510. doi:10.1371/journal.pcbi.1005510.
- [27] P. Virtanen, et al., SciPy 1.0: Fundamental algorithms for scientific computing in python, *Nat. Methods* 17 (2020) 261–272. doi:10.1038/s41592-019-0686-2.
- [28] R. H. Byrd, P. Lu, J. Nocedal, C. Zhu, A limited memory algorithm for bound constrained optimization, *SIAM J. Sci. Comput.* 16 (1995) 1190–1208. doi:10.1137/0916069.